



IT SIMPLERA INSTITUTE
Machine Learning Internship Program

Week 5 Task Documentation

Natural Language Processing & Sentiment Analysis

TF-IDF | Logistic Regression | Multinomial Naive Bayes | Streamlit Dashboard

Dataset: Amazon Product Reviews Dataset (Kaggle)

1. Objective of the Task

This week's task moves from unsupervised learning into Natural Language Processing (NLP), focused on teaching a machine to understand human text. The goal was to work with real Amazon customer reviews, clean the raw text, convert it into a numerical format that a model can learn from, and build a classifier that predicts whether a review is positive or negative.

Sentiment analysis of this kind is widely used across industries — it powers customer feedback systems, product review platforms, chatbots, and recommendation engines by automatically surfacing how customers feel about a product or service, without requiring manual reading of every review.

The task was split into two parts:

- Part 1 — NLP preprocessing, feature extraction, and training/evaluating text classification models in Google Colab.
- Part 2 — Wrapping the trained model into an interactive Streamlit dashboard for real-time sentiment prediction.

2. Dataset Overview

Property	Details
Name	Amazon Product Reviews Dataset
Source	Kaggle — dhruvlotia/amazon-review-sentiment-analysis
Records	3,150 customer reviews (3,149 after cleaning/deduplication of nulls)
Key columns	verified_reviews (review text), feedback (1 = positive, 0 = negative)
Class balance	2,893 positive reviews vs. 256 negative reviews — a significantly imbalanced dataset

3. Approach & Methodology

Part 1 — NLP Preprocessing & Model Training

- Load the dataset and inspect the review text and label columns; check the class distribution of sentiments.
- Clean the raw text: lowercase conversion, removal of URLs/HTML tags, removal of punctuation and numeric characters, removal of stopwords and generic filler words, and dropping single-character tokens.
- Generate a Word Cloud for positive reviews and a separate one for negative reviews, and display them together side-by-side.
- Convert the cleaned text into numerical features using TF-IDF vectorization (with an n-gram range of 1–2).
- Split the data into training and testing sets (80/20, stratified) and train two classification models: Logistic Regression and Multinomial Naive Bayes.
- Address the severe class imbalance (92% positive vs. 8% negative) by oversampling the minority class in the training set and using `class_weight="balanced"` for Logistic Regression.
- Evaluate each model using accuracy, precision, recall, and F1-score, and visualize results with a Confusion Matrix for each model.

- Compare model performance in a chart and save the best-performing model along with its vectorizer for use in Part 2.

Part 2 — Streamlit Dashboard

- Build a three-page Streamlit dashboard accessible via a sidebar: Home, Data Overview, and Sentiment Predictor.
- Home page introduces the project and the dataset.
- Data Overview page displays the sentiment class distribution chart and the two word clouds from Part 1.
- Sentiment Predictor page accepts a free-text review from the user, loads the saved model and vectorizer, applies the same cleaning pipeline used in training, and returns the predicted sentiment along with a confidence score.

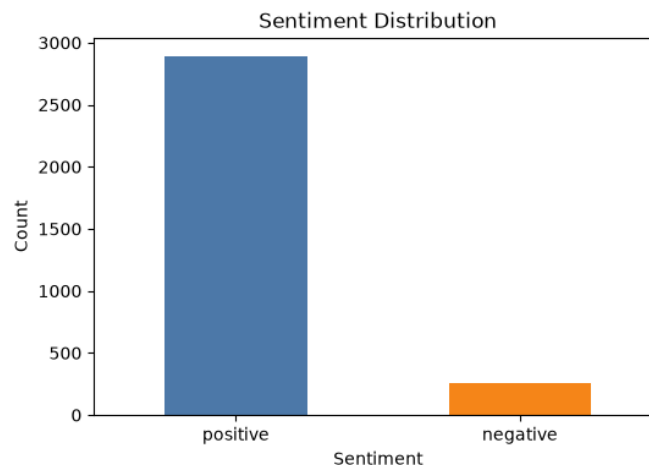
4. Implementation & Results

4.1 Data Loading & Inspection

The dataset was loaded and inspected: 3,150 rows and 5 columns (rating, date, variation, verified_reviews, feedback). The review text column (verified_reviews) and label column (feedback) were identified. After dropping rows with missing values, 3,149 usable records remained.

	rating	date	variation	verified_reviews	feedback
0	5	31-Jul-18	Charcoal Fabric	Love my Echo!	1
1	5	31-Jul-18	Charcoal Fabric	Loved it!	1
2	4	31-Jul-18	Walnut Finish	Sometimes while playing a game, you can answer...	1
3	5	31-Jul-18	Charcoal Fabric	I have had a lot of fun with this thing. My 4 ...	1
4	5	31-Jul-18	Charcoal Fabric	Music	1

The class distribution revealed a strong imbalance: 2,893 positive reviews versus only 256 negative reviews.



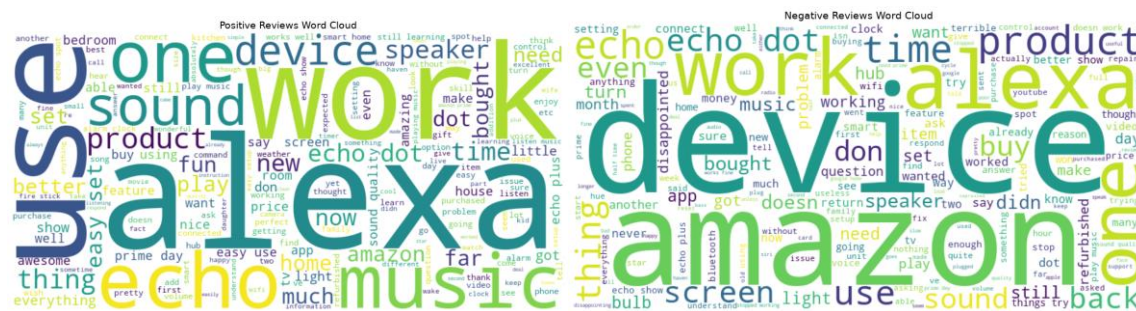
4.2 Text Cleaning

A cleaning function was applied to every review: text was lowercased, URLs and HTML tags were stripped, non-alphabetic characters were removed, and a custom stopwords list (including generic filler and overly generic sentiment words) was used to filter tokens. Single-character tokens were also discarded, producing a cleaned_review column used for all downstream modeling.

	verified_reviews	cleaned_review
0	Love my Echo!	echo
1	Loved it!	
2	Sometimes while playing a game, you can answer...	sometimes while playing game answer question c...
3	I have had a lot of fun with this thing. My 4 ...	lot fun thing yr old learns about dinosaurs co...
4	Music	music

4.3 Word Cloud Generation

Word clouds were generated separately for positive and negative reviews and displayed together in a single side-by-side figure, making it easy to visually compare the most frequent terms associated with each sentiment class.



4.4 Feature Extraction — TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) was selected over a simple Bag-of-Words representation because it down-weights words that occur frequently across the entire corpus (and therefore carry little discriminative value) while up-weighting terms that are more specific to individual reviews. An n-gram range of (1, 2) was used so that both single words and two-word phrases (e.g. "not good") are captured, which is particularly useful for sentiment-bearing expressions.

4.5 Train-Test Split & Handling Class Imbalance

The data was split into training and testing sets using an 80/20 stratified split, preserving the original class ratio in the test set. Because the raw training data was heavily skewed toward positive reviews, initial models learned to predict "positive" almost universally, achieving high accuracy (92%) but completely failing to detect negative reviews (0% recall on the negative class).

To correct this, the minority (negative) class in the training set was oversampled to match the majority class, and Logistic Regression was additionally configured with `class_weight="balanced"` to further penalize misclassification of the minority class during training. The test set was left untouched to ensure evaluation reflected real-world class distribution.

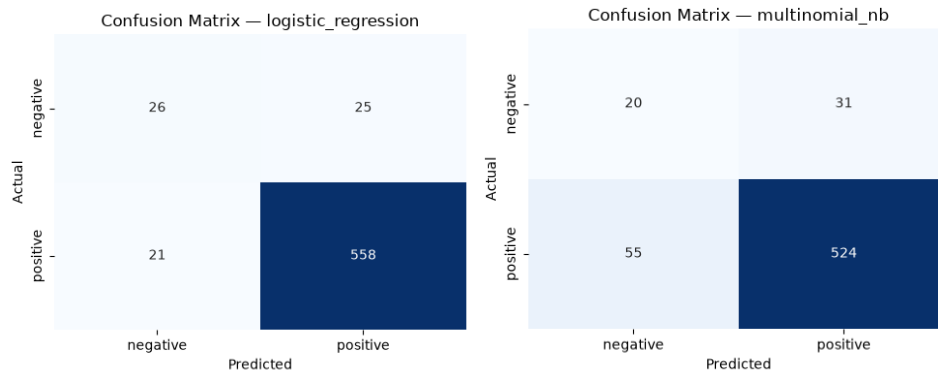
4.6 Model Training & Evaluation

Two classifiers were trained on the TF-IDF features: Logistic Regression and Multinomial Naive Bayes. Performance after addressing class imbalance:

Metric (negative class)	Logistic Regression	Multinomial Naive Bayes
Precision	0.55	0.27
Recall	0.51	0.39

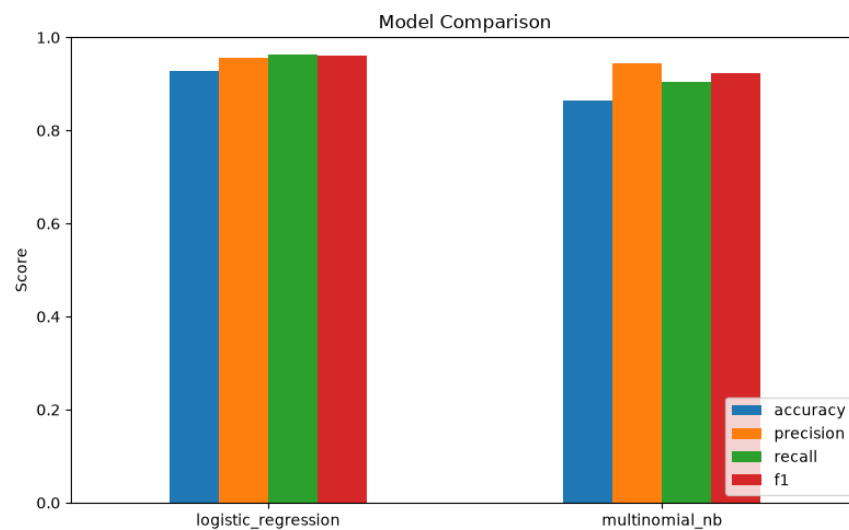
F1-score	0.53	0.32
Overall Accuracy	0.93	0.86

Logistic Regression clearly outperformed Multinomial Naive Bayes on the minority (negative) class after rebalancing, while maintaining strong performance on the majority (positive) class (precision 0.96, recall



4.7 Model Comparison & Best Model Selection

Accuracy, precision, recall, and F1-score were compared across both models in a single bar chart. Logistic Regression was selected as the best-performing model based on F1-score, and was saved along with its TF-IDF vectorizer for use in the Streamlit dashboard.



4.8 Streamlit Dashboard

The saved model and vectorizer were integrated into a three-page Streamlit application.

Home page — project introduction and dataset description.

Go to

- Home
- Data Overview
- Sentiment Predictor

Amazon Review Sentiment Analysis

This project uses NLP to analyze Amazon customer reviews and predict whether a review is positive or negative.

The dataset contains customer feedback text and sentiment labels, which are used to train and evaluate text classifiers.

The app uses a trained TF-IDF model and a cleaned text pipeline for prediction.

Data Overview page — sentiment class distribution and word clouds.

Go to

- Home
- Data Overview
- Sentiment Predictor

Data Overview

Review sentiment distribution and word clouds from the training notebook are shown below.

Sentiment Predictor page — free-text review input with predicted sentiment and confidence score.

Go to

- Home
- Data Overview
- Sentiment Predictor

Sentiment Predictor

Type a customer review and the app will predict whether it is positive or negative.

Enter a review

This product stopped working after just two days. Complete waste of money.

Predict Sentiment

Predicted sentiment: negative

Confidence

95.4%

The dashboard was tested with a clearly negative review — "This product stopped working after just two days. Complete waste of money." — which the model correctly classified as negative with 95.4% confidence, confirming the class-imbalance handling was effective end-to-end.

5. Comparison Report & Conclusion

This task provided hands-on experience with the full NLP pipeline for sentiment classification: text cleaning, TF-IDF feature extraction, model training, and — critically — recognizing and correcting a severe class imbalance that initially caused both models to ignore the negative class entirely.

- Logistic Regression is recommended as the production model: it achieved a substantially better balance between the positive and negative classes after rebalancing, with an overall accuracy of 93% and a negative-class F1-score of 0.53.
- Multinomial Naive Bayes remains a fast, lightweight baseline, but underperforms on the minority class in this dataset.
- The core lesson from this task was that accuracy alone is a misleading metric on imbalanced text data — per-class precision, recall, and F1-score, combined with explicit imbalance-handling techniques, were necessary to build a genuinely useful sentiment classifier.

The resulting Streamlit dashboard makes the trained model directly usable: any customer review can be typed in and classified in real time, with a transparent confidence score, directly supporting practical use cases such as customer feedback triage and review monitoring.